

Lab 8 (08 Oct)

Q1 (40%)

Heaps.

We have discussed binary heaps in the class. The assignment is to implement a generalization of BinaryHeaps to implement 3-ary Heaps along with the routines discussed in the class. You may use the code for Binary Heaps, given in the .zip file.

implement a Priority Queue based on a 3-ary Heap. The priority queue supports 3 operations:

- `Insert` - Insert new elements into the Priority Queue (Keys will be distinct)
- `DeleteMax` - Remove the largest key. Ignore if the PQ is empty.
- `Maximum` - Print the largest Key, or "Empty" if PQ is empty
- `BuildHeap` : Given an array of distinct integers builds a 3-ary heap – No test cases will be given.

You may start from the binary MaxHeap implementation in class and change it accordingly.

A list of operations is given as input. You must perform those operations sequentially, and output the result of the Maximum operations.

Input Format

The first line of the input contains a single integer n , the number of operations. The following n lines,

each of which is one of the following:

- `Insert xyz` , where `xyz` is an integer.
- `DeleteMax`
- `Maximum`

Output Format

For every Maximum operation, print the maximum key in a new line. Each line contains a single integer. If the heap is empty when Maximum is performed, print Empty. If the heap is empty when DeleteMax is performed, do not print anything.

Constraints:

- Number of operations, $N \leq 2000$.
- $-200 \leq \text{Keys} \leq 200$.

Example:

Input:

10

Maximum

Insert 35

Insert 100
Maximum
DeleteMax
Insert 25
Maximum
Insert 75
Insert 65
Maximum

Output:

Empty
100
35
75

Q2(30%)

This question builds on the heap data structure developed in Q1.

The main task is to develop an algorithm that prints the k largest elements in an array using heaps. You may also assume that no key appears more than once in the array. For full credits, the algorithm should take $O(k \log k + n)$ time (The $O(n)$ time should build the heap.) A $(k \log n + n)$ algorithm will fetch at most 75% of the credits for the problem and a $O(n \log n)$ algorithm can fetch a maximum of 50% of the credits.

Note: `priority_queue` from STL library may be used. You can also use a template version of your heap developed in Q1.

Input:

N - number of elements in the array.
 K - number of largest elements to find.
 N integers

Output:

Print the **K largest elements** among the N integers, in **descending order** (largest to smallest). Each element should be separated by a single space.

Constraints

$1 \leq N \leq 10^6$

$1 \leq K \leq 100$

Example:

Input :

10

3

15 3 9 8 12 20 5 2 7 11

Output :

20 15 12

Q3(30%)

In this assignment you are required to implement the Huffman Coding algorithm. You may use the Binary Tree and the Heap Data structures you have built during this lab or previous lab sessions. If needed you may also use `std::priority_queue`.

Input will be a string (that may include blank spaces). You need to run the Huffman encoding algorithm to obtain an optimal binary tree representing the Huffman encoding of the given string. You are expected to provide an algorithm with the best possible running time (within $O(N)$). For a non-optimal solution, the maximum credit will be 75%.

Frequency Ordering:

- Nodes with smaller frequency go to the **left child**.
- Nodes with larger frequency go to the **right child**.

Tiebreaker for Equal Frequencies:

- If two nodes have the **same frequency**, the node with the **smaller ASCII value** goes to the **left**, and the other goes to the **right**.

Internal Node ASCII Value:

- For internal nodes, assign the node the **smallest ASCII value** among all its leaf descendants.

Code Assignment:

- When generating Huffman codes, assign **'0'** to the **left child** and **'1'** to the **right child**.

Constraints:

N, length of the string $\leq 10^6$

Alphabets: A-Z a-z and ` , ' , ` : ' , ` . ' ` ` (blank space)

Input:

First line: N

Second line: A string of length N (including blank spaces)

Output:

Code assigned by the Huffman encoding on each of the characters in the input string. Each line must contain the character followed by a blank and then the codeword for the character .
(Output should be sorted(ascending ordered) by character ASCII value).

Example: (TAs please fill up)

Input:

13

AABACADABRA.

Output:

1010
 . 1011
 A 0
 B 110
 C 1110
 D 1111
 R 100

Bonus (5 marks, capped globally to 5%)

(Will be evaluated only if you complete Q1, Q2 and Q3, passing all open test cases)

During your favorite festival, you receive a box of N laddoos. The laddoos are of different sizes, $A[1], A[2], \dots, A[N]$. You have K moves, and in each move you can pick any one laddoo and eat half of it. Specifically, you eat $\lfloor A[i]/2 \rfloor$ of the chosen laddoo i, with $\lceil A[i]/2 \rceil$ laddoo remaining. Your goal is code up a strategy to eat as much total amount of laddoo as possible (i.e., total amount across the K moves).

You can use `std::priority queue`. Expected time complexity is $O(N + K \log N)$.

Input Format

The first line of the input contains (N, K) two space separated integers N (number of laddoos) and K

(number of moves). The second line contains N space separated integers, which are the sizes of the N laddoos.

Output Format

Print a single integer value, that is the maximum amount of laddoo you can eat (summed across all moves).

Constraints:

- Number of laddoos, $N \leq 1000$.
- $0 \leq A[i] \leq 10^4$.

Example 1:**Input:**

54

8 10 8 8 8

Output:

17

Explanation: We eat half of the largest 4 laddoos.

Example 2:**Input:**

44

21 8 5 3

Output:

22

Explanation: $(21, 8, 5, 3) \rightarrow (11, 8, 5, 3) \rightarrow (6, 8, 5, 3) \rightarrow (6, 4, 5, 3) \rightarrow (3, 4, 5, 3)$